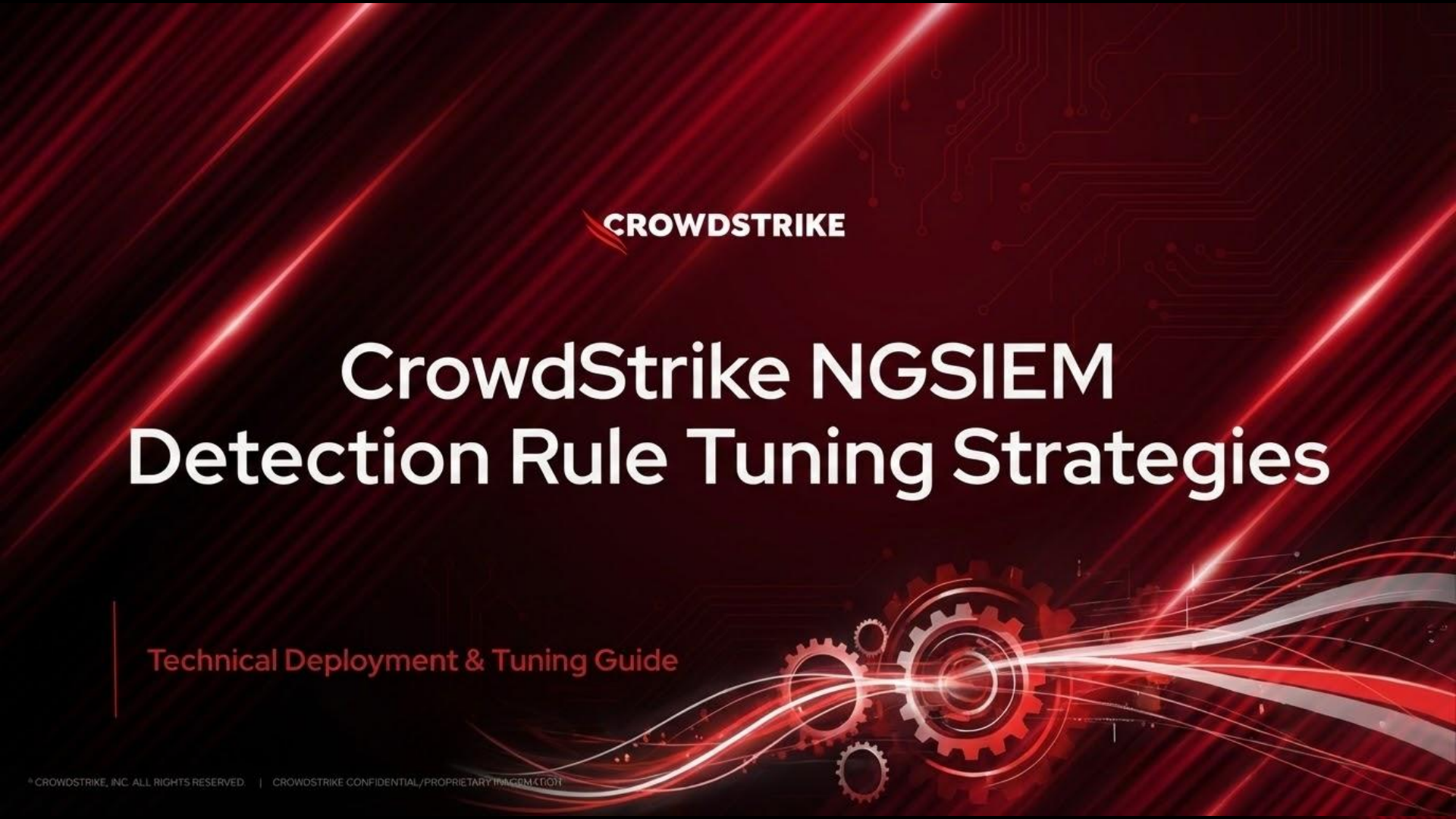
CROWDSTRIKE

CrowdStrike NGSIEM Detection Rule Tuning Strategies

Technical Deployment & Tuning Guide



CrowdStrike NGSIEM Detection Rule Tuning Strategies

Technical Deployment & Tuning Guide

Purpose

False Positives and rule tuning are expected parts of your organization's detection development. These will be activities that your teams need to take into account. How and to what extent are up to you. When we consider a security operations detection development model, we can see that it is meant to be an iterative process

The purpose of this resource is to address false positives seen within NGSIEM detection rules. Other modules, false negatives, etc. will follow additional processes.

Purpose

False Positives and rule tuning are expected parts of your organization's detection development. These will be activities that your teams need to take into account. How and to what extent are up to you. When we consider a security operations detection development model, we can see that it is meant to be an iterative process

The purpose of this resource is to address false positives seen within NGSIEM detection rules. Other modules, false negatives, etc. will follow additional processes.

DETECTION DEVELOPMENT LIFECYCLE

Modern SOCs leverage agile, iterative approaches for content development.

PLAN

- Conceptual framework for the use case is established.
- Requirements are gathered, stakeholders are identified and engaged, and alignment with business objectives is ensured.

BUILD

- The abstract use case elements are transformed into concrete implementations.
- Documentation is created, technical requirements are specified, and the use case is prepared for operational deployment.

CHANGE

- Addresses necessary modifications to keep use cases relevant and effective.
- Includes the offloading process, which ensures proper removal from the framework.

RUN

- UC becomes fully operational within daily security operations.

Source: FI-ISAC MaGMA Use Case Framework

CROWDSTRIKE

Detection Development Lifecycle

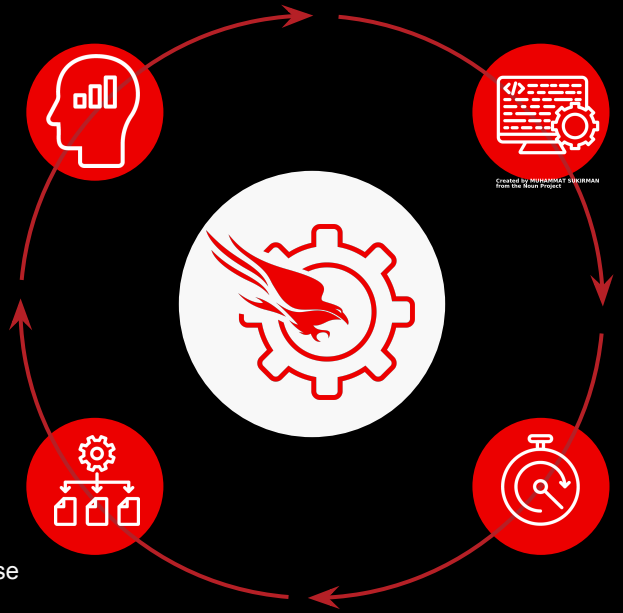
Modern SOCs leverage agile, iterative approaches for content development.

PLAN

- Conceptual framework for the use case is established.
- Requirements are gathered, stakeholders are identified and engaged, and alignment with business objectives is ensured.

BUILD

- The abstract use case elements are transformed into concrete implementations.
- Documentation is created, technical requirements are specified, and the use case is prepared for operational deployment.



CHANGE

- Addresses necessary modifications to keep use cases relevant and effective.
- Includes the offloading process, which ensures proper removal from the framework.

RUN

- UC becomes fully operational within daily security operations.

Source: FI-ISAC MaGMA Use Case Framework

KEY PRINCIPLES DRIVE YOUR DEVELOPMENT PROCESS

Iterative by Design

Feedback Driven

Agile Mindset

Lifecycle Management

Tuning Strategies for Net-New Rules

Scenario 1

A threat actor has **compromised administrative credentials and is accessing production systems during off-hours** to avoid detection. The attacker logs into multiple Linux and Windows servers between 2-4 AM on weekdays and throughout weekend hours, performing reconnaissance and data exfiltration while legitimate administrators are offline. However, **legitimate automated service accounts (backup services, monitoring agents, and patch automation) also operate during these hours, requiring careful filtering** to distinguish malicious interactive logons from expected service account activity and reduce false positives in the detection logic.

Inline Filters

```
| #event_simpleName=UserLogon UserIsAdmin=1
| hour:=formatTime("%H")
| day_name:=formatTime("%A")
| case {
    hour<"07" | after_hours:=true;
    hour>"19" | after_hours:=true;
    day_name="Saturday" | after_hours:=true;
    day_name="Sunday" | after_hours:=true;
    * | after_hours:=false;
  }
| after_hours=true
| groupBy([UserName, hour, day_name], function=[
    count(as=logon_count),
    collect(ComputerName, RemoteAddressIP4)])
```

False positive service accounts need to be removed from results.

UserName	hour	day_name	logon_count	ComputerName	RemoteAddressIP4
admin_compromised	15	Sunday	1	prod-backup-srv	203.0.113.45
admin_compromised	20	Monday	1	prod-web-02	203.0.113.45
admin_compromised	22	Monday	1	prod-app-03	203.0.113.45
svc_monitoring_agent	3	Thursday	1	prod-web-02	10.0.10.50
svc_monitoring_agent	22	Sunday	1	prod-app-03	10.0.10.50
svc_nightly_backup	1	Friday	1	prod-file-srv	10.0.5.100
svc_nightly_backup	1	Thursday	2	prod-db-01 prod-backup-srv	10.0.5.100
svc_patch_automation	2	Monday	1	prod-app-03	10.0.20.75
svc_patch_automation	2	Sunday	1	prod-web-02	10.0.20.75

```
| #event_simpleName=UserLogon UserIsAdmin=1
// FILTERING STRATEGY: Exclude known service accounts
| !in(field=UserName, values=[svc_patch_automation, svc_nightly_backup, svc_monitoring_agent])
| hour:=formatTime("%H")
| day_name:=formatTime("%A")
| case {
    hour<"07" | after_hours:=true;
    hour>"19" | after_hours:=true;
    day_name="Saturday" | after_hours:=true;
    day_name="Sunday" | after_hours:=true;
    * | after_hours:=false;
  }
| after_hours=true
| groupBy([UserName, hour, day_name], function=[
    count(as=logon_count),
    collect(ComputerName, RemoteAddressIP4)])
```

False positive service accounts have been removed from results.

UserName	hour	day_name	logon_count	ComputerName	RemoteAddressIP4
admin_compromised	2	Thursday	2	prod-web-02 prod-db-01	203.0.113.45
admin_compromised	3	Thursday	1	prod-app-03	203.0.113.45
admin_compromised	5	Thursday	1	prod-db-01	203.0.113.45
admin_compromised	14	Sunday	1	prod-file-srv	203.0.113.45
admin_compromised	15	Sunday	1	prod-backup-srv	203.0.113.45
admin_compromised	20	Monday	1	prod-web-02	203.0.113.45
admin_compromised	22	Monday	1	prod-app-03	203.0.113.45

Filtering via Saved Searches

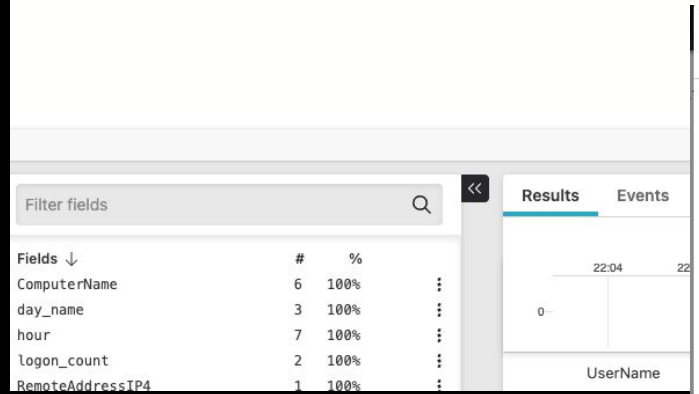
```
| #event_simpleName=UserLogon UserIsAdmin=1
| hour:=formatTime("%H")
| day_name:=formatTime("%A")
| case {
    hour<"07" | after_hours:=true;
    hour>"19" | after_hours:=true;
    day_name="Saturday" | after_hours:=true;
    day_name="Sunday" | after_hours:=true;
    * | after_hours:=false;
  }
| after_hours=true
| groupBy([UserName, hour, day_name], function=[
    count(as=logon_count),
    collect(ComputerName, RemoteAddressIP4)])
```

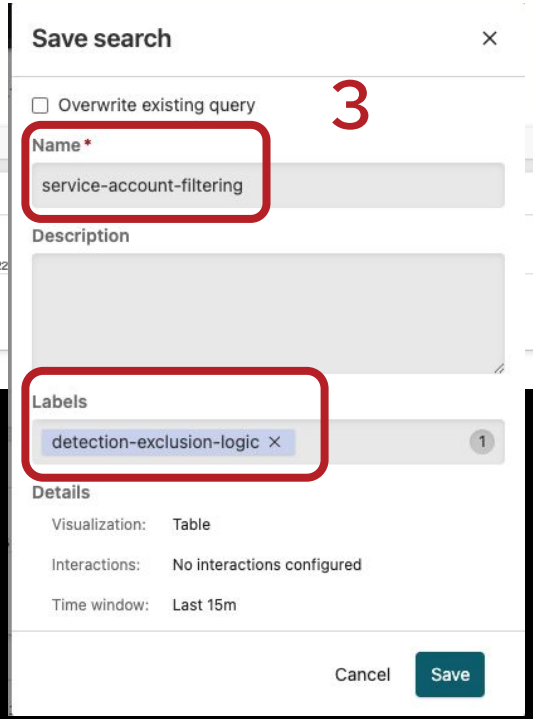
False positive service accounts need to be removed from results.

UserName	hour	day_name	logon_count	ComputerName	RemoteAddressIP4
admin_compromised	15	Sunday	1	prod-backup-srv	203.0.113.45
admin_compromised	20	Monday	1	prod-web-02	203.0.113.45
admin_compromised	22	Monday	1	prod-app-03	203.0.113.45
svc_monitoring_agent	3	Thursday	1	prod-web-02	10.0.10.50
svc_monitoring_agent	22	Sunday	1	prod-app-03	10.0.10.50
svc_nightly_backup	1	Friday	1	prod-file-srv	10.0.5.100
svc_nightly_backup	1	Thursday	2	prod-db-01 prod-backup-srv	10.0.5.100
svc_patch_automation	2	Monday	1	prod-app-03	10.0.20.75
svc_patch_automation	2	Sunday	1	prod-web-02	10.0.20.75

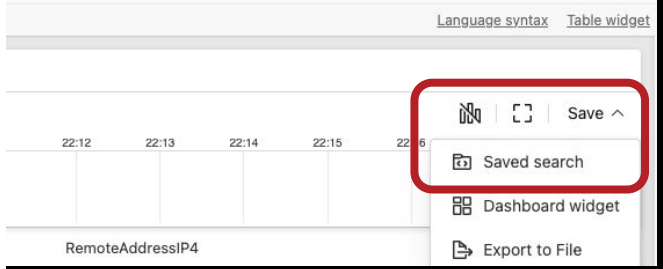
```
// FILTERING STRATEGY: Exclude known service accounts
| !in(field=UserName, values=[svc_patch_automation, svc_nightly_backup, svc_monitoring_agent])

```

1

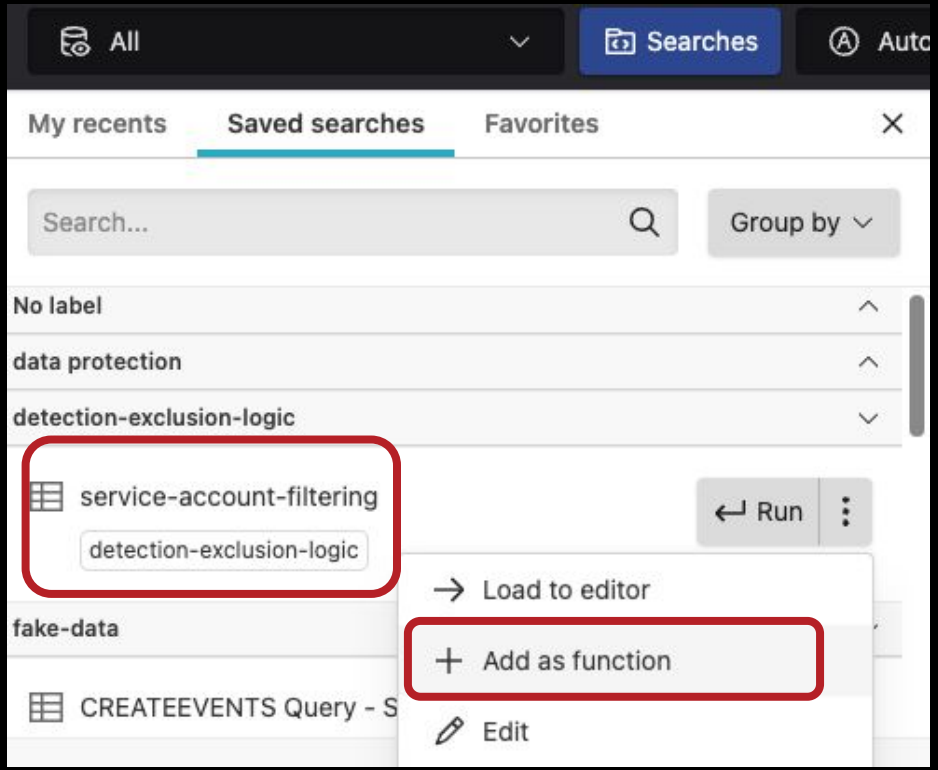




2



CROWDSTRIKE



Add your saved search as a function into your detection logic.

```
| #event_simpleName=UserLogon UserIsAdmin=1
| $"service-account-filtering"()
| hour:=formatTime("%H")
| day_name:=formatTime("%A")
| case {
    hour<"07" | after_hours:=true;
    hour>"19" | after_hours:=true;
    day_name="Saturday" | after_hours:=true;
    day_name="Sunday" | after_hours:=true;
    * | after_hours:=false;
  }
| after_hours=true
| groupBy([UserName, hour, day_name], function=[
    count(as=logon_count),
    collect(ComputerName, RemoteAddre
```

False positive service accounts have been removed from results.

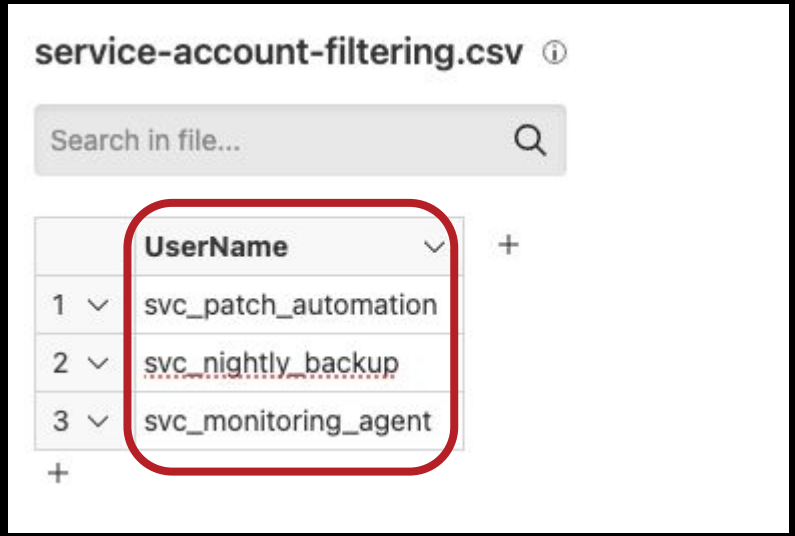
	UserName	hour	day_name	logon_count	ComputerName	RemoteAddressIP4
⋮	admin_compromised	2	Thursday	2	prod-web-02 prod-db-01	203.0.113.45
⋮	admin_compromised	3	Thursday	1	prod-app-03	203.0.113.45
⋮	admin_compromised	5	Thursday	1	prod-db-01	203.0.113.45
⋮	admin_compromised	14	Sunday	1	prod-file-srv	203.0.113.45
⋮	admin_compromised	15	Sunday	1	prod-backup-srv	203.0.113.45
⋮	admin_compromised	20	Monday	1	prod-web-02	203.0.113.45
⋮	admin_compromised	22	Monday	1	prod-app-03	203.0.113.45

Filtering via Lookup Files

```
| #event_simpleName=UserLogon UserIsAdmin=1
| hour:=formatTime("%H")
| day_name:=formatTime("%A")
| case {
    hour<"07" | after_hours:=true;
    hour>"19" | after_hours:=true;
    day_name="Saturday" | after_hours:=true;
    day_name="Sunday" | after_hours:=true;
    * | after_hours:=false;
  }
| after_hours=true
| groupBy([UserName, hour, day_name], function=[
    count(as=logon_count),
    collect(ComputerName, RemoteAddressIP4)])
```

False positive service accounts need to be removed from results.

UserName	hour	day_name	logon_count	ComputerName	RemoteAddressIP4
admin_compromised	15	Sunday	1	prod-backup-srv	203.0.113.45
admin_compromised	20	Monday	1	prod-web-02	203.0.113.45
admin_compromised	22	Monday	1	prod-app-03	203.0.113.45
svc_monitoring_agent	3	Thursday	1	prod-web-02	10.0.10.50
svc_monitoring_agent	22	Sunday	1	prod-app-03	10.0.10.50
svc_nightly_backup	1	Friday	1	prod-file-srv	10.0.5.100
svc_nightly_backup	1	Thursday	2	prod-db-01 prod-backup-srv	10.0.5.100
svc_patch_automation	2	Monday	1	prod-app-03	10.0.20.75
svc_patch_automation	2	Sunday	1	prod-web-02	10.0.20.75



Create a lookup file with your filtering/exclusion values.

```
| #event_simpleName=UserLogon UserIsAdmin=1
| !match(file="service-account-filtering.csv", column=UserName, field=UserName)
| hour:=formatTime("%H")
| day_name:=formatTime("%A")
| case {
    hour<"07" | after_hours:=true;
    hour>"19" | after_hours:=true;
    day_name="Saturday" | after_hours:=true;
    day_name="Sunday" | after_hours:=true;
    * | after_hours:=false;
  }
| after_hours=true
| groupBy([UserName, hour, day_name], function=[
    count(as=logon_count),
    collect(ComputerName, RemoteAddressIP4)])
```

False positive service accounts have been removed from results.

			_name	logon_count	ComputerName	RemoteAddressIP4
⋮	admin_compromised	2	Thursday	2	prod-web-02 prod-db-01	203.0.113.45
⋮	admin_compromised	3	Thursday	1	prod-app-03	203.0.113.45
⋮	admin_compromised	5	Thursday	1	prod-db-01	203.0.113.45
⋮	admin_compromised	14	Sunday	1	prod-file-srv	203.0.113.45
⋮	admin_compromised	15	Sunday	1	prod-backup-srv	203.0.113.45
⋮	admin_compromised	20	Monday	1	prod-web-02	203.0.113.45
⋮	admin_compromised	22	Monday	1	prod-app-03	203.0.113.45